

Functors

Programming Languages

Sujit Kumar Chakrabarti

IITB

Functors

What is a Functor?

- ▶ Can be thought of as a parameterised module
- ▶ Here's how a functor is defined:

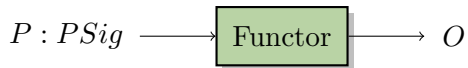
```
module F (P : PSig) =  
  struct  
    ...  
  end
```

- ▶ *F* is the functor name.
- ▶ *P* is the name of parameter module.
- ▶ *PSig* is the module interface which *P* promises to comply to.

Functors

What is a Functor?

- ▶ Can also be thought of as a Function that takes modules as inputs and returns a module as output



- ▶ Comparison to functions is limited because modules aren't first class objects in OCaml.

Functors

A Module Signature for Comparable data

```
module type COMP =  
sig  
  type t  
  val (<) : t -> t -> bool  
  val (>) : t -> t -> bool  
  val (=) : t -> t -> bool  
  val string_of_t : t -> bytes (* string conversion function for t *)  
end
```

Declaring that there will be type signatures in the module

using t as a type variable

parenthesis around function/operators means that we intend to use them as infix operators

Functors

A Module Implementation for Comparable data - Integer Comparable

Indicates that this module implements (or complies with) the COMP module signature.
It also specifies that the abstract type `t` of COMP will take the meaning of 'int' in the context of this module.

```
module IntComp : (COMP with type t = int) =  
  struct  
    type t = int  
  
    let (<) = (<)  
    let (>) = (>)  
    let (=) = (=)  
    let string_of_t = string_of_int  
  end
```

Means that these functions (<), (>) and (=) are being assigned their usual meaning.
Notice the role of functions as first class objects here which makes the syntax lightweight and your program more efficient at runtime.

Functors

A Module Signature for Pair

This is the module signature that the modules created by application of the Functor will adhere to.

```
module type PAIR =
```

```
sig
```

```
  type elt
```

```
  type t
```

Defines these type variables that will be used in the type signatures of all declarations in this module signature.

```
  val newpair : elt -> elt -> t
```

```
  val first : 'a * 'b -> 'a
```

```
  val second : 'a * 'b -> 'b
```

```
  val ascending : elt * elt -> elt * elt
```

```
  val descending : elt * elt -> elt * elt
```

```
  val string_of_pair : t -> bytes
```

```
end
```

t denotes the data structure (container) type
elt denotes the element type.

Functors

A Functor to Create Type Specific Pair

Parameter module name is C, and it complies with the module signature COMP.

Module signature to be complied to by the returned module.
It is often not necessary to specify this.

Type sharing to stitch together two interacting modules.

```
module Pair (C : COMP) : PAIR
  with type elt = C.t with type t = C.t * C.t =
  struct
    type elt = C.t
    type t = elt * elt

    let newpair (f : C.t) (s : C.t) = (f, s)
    let first (f, _) = f
    let second (_, s) = s
    let order (f, s) ~func = if (func f s) then (f, s) else (s, f)
    let ascending = order ~func:C.<
    let descending = order ~func:C.>

    let string_of_pair (f, s) = "(" ^ (C.string_of_t f) ^ ", " ^
      (C.string_of_t s) ^ ")"
  end
```

Here we are concretising the types which were left abstract in PAIR signature.

Functors

A Functor to Create Type Specific Pair

```
module Pair (C : COMP) : PAIR
  with type elt = C.t with type t = C.t * C.t =
  struct
    type elt = C.t
    type t = elt * elt
```

Examples of the fact that OCaml allows explicit typing. Not necessary here.

```
  let newpair (f : C.t) (s : C.t) = (f, s)
  let first (f, _) = f
  let second (_, s) = s
  let order (f, s) ~func = if (func f s) then (f, s) else (s, f)
  let ascending = order ~func:C.(<)
  let descending = order ~func:C.(>)

  let string_of_pair (f, s) = "(" ^ (C.string_of_t f) ^ ", " ^
    (C.string_of_t s) ^ ")"

end
```

Such a function is necessary as we want to keep the container implementation hidden from the module interface.

Functors

A Functor to Create Type Specific Pair

```
module Pair (C : COMP) : PAIR
  with type elt = C.t with type t = C.t * C.t =
  struct
    type elt = C.t
    type t = elt * elt
```

Examples of the fact that OCaml allows explicit typing. Not necessary here.

```
  let newpair (f : C.t) (s : C.t) = (f, s)
  let first (f, _) = f
  let second (_, s) = s
  let order (f, s) ~func = if (func f s) then (f, s) else (s, f)
  let ascending = order ~func:C.(<)
  let descending = order ~func:C.(>)
```

Declaring a formal parameter to be named.

This is how a named parameter is used in a function call.

```
  let string_of_pair (f, s) = "(" ^ (C.string_of_t f) ^ ", " ^
    (C.string_of_t s) ^ ")"
end
```

Both functions ascending and descending are created by partial application of order function.

Functors

A Functor to Create Type Specific Pair

```
module Pair (C : COMP) : PAIR
  with type elt = C.t with type t = C.t * C.t =
struct
  type elt = C.t
  type t = elt * elt
```

Notice how we refer to members of the parameter module C in the definitions within the functor.

```
  let newpair (f : C.t) (s : C.t) = (f, s)
  let first (f, _) = f
  let second (_, s) = s
  let order (f, s) ~func = if (func f s) then (f, s) else (s, f)
  let ascending = order ~func:C.(<)
  let descending = order ~func:C.(>)

  let string_of_pair (f, s) = "(" ^ (C.string_of_t f) ^ ", " ^
    (C.string_of_t s) ^ ")"
end
```

Functors

Integer Specific Pair by Application of Functor

```
module IntPair = Pair(IntComp)
```

Generated module

Functor

Argument Module

Simple!

Functors

Test Case for IntPair

Using fully qualified names (i.e. <modulename>.<membername>) to access module members is generally a good idea. But there are some disadvantages:

- More typing
- May hamper readability

```
open IntPair
```

In such cases, you can open a module and access its contents directly.

CAUTION: Use sparingly as it can cause name clashes.

```
let tint () =  
  let p1 = newpair 1 2 in  
  
  print_endline (string_of_pair p1);  
  print_endline ("first = " ^ (IntComp.string_of_t (first p1)));  
  print_endline ("Ascending = " ^ string_of_pair (ascending p1));  
  print_endline ("Descending = " ^ string_of_pair (descending p1))
```

```
utop # tint ();;  
(1, 2)  
first = 1  
Ascending = (1, 2)  
Descending = (2, 1)  
- : unit = ()
```

Note that this feature has nothing to do with functors. It's more about modules.

Functors

A Module Implementation for Comparable data - Date Comparable

```
type date = {  day : int; month : int; year : int }
```

```
module DateComp : ((COMP with type t = date) =  
struct
```

Sharing type COMP.t as date.

```
  type t = date
```

Note that it explicitly complies with
COMP module signature.

```
  let (<) d1 d2 =  
    if      d1.year < d2.year then true  
    else if d1.year > d2.year then false  
    else if d1.month < d2.month then true  
    else if d1.month > d2.month then false  
    else if d1.day < d2.day then true  
    else false
```

```
  let (=) d1 d2 = d1.day = d2.day && d1.month = d2.month && d1.year = d2.year  
  let (>) d1 d2 = (not (d1 = d2)) && (not (d1 < d2))
```

```
  let string_of_t d = "Date (" ^ (string_of_int d.day) ^  
                        ", " ^ (string_of_int d.month) ^  
                        ", " ^ (string_of_int d.year) ^ ")"
```

```
end
```

This module is created an another example
of a parameter module to the Pair functor.
This module is a library of a simple date
datatype.

Functors

Date Specific Pair by Application of Functor

```
module DatePair = Pair(DateComp)
```

Generated module

Functor

Argument Module

Simple!

Functors

Date Specific Pair by Application of Functor

```
let tdate () =  
  let d1 = {day = 21; month = 3; year = 2022}  
  and d2 = {day = 1; month = 4; year = 2022} in  
  let open DatePair in  
  let p1 = newpair d1 d2 in  
  print_endline (string_of_pair p1);  
  print_endline ("first = " ^ (DateComp.string_of_t (first p1)));  
  print_endline ("Ascending = " ^ string_of_pair (ascending p1));  
  print_endline ("Descending = " ^ string_of_pair (descending p1))
```

Opening a module locally. Much safer than globally opening a module. Still to be used sparingly.

```
utop # tdate ();;  
(Date (21, 3, 2022), Date (1, 4, 2022))  
first = Date (21, 3, 2022)  
Ascending = (Date (21, 3, 2022), Date (1, 4, 2022))  
Descending = (Date (1, 4, 2022), Date (21, 3, 2022))  
- : unit = ()
```